```
  database.updateUnique(
      "INSERT INTO user_roles(space_id, user_id, role_id) " +
              "VALUES(?, ?, ?)", spaceId, owner, "owner");
```

Updating `addMember` involves a little more code, because you should ensure that you validate the new role. Add the following line to the top of the class to define the valid roles:

```
  private static final Set<String> DEFINED_ROLES =
          Set.of("owner", "moderator", "member", "observer");
```

You can now update the implementation of the `addMember` method to be role-based instead of permission-based, as shown in listing 8.7. First extract the desired role from the request and ensure it is a valid role name. You can default to the `member` role if none is specified as this is the normal role for most members. It is then simply a case of inserting the role into the `user_roles` table instead of the old `permissions` table and returning the assigned role in the response.

Listing 8.7 Adding new members with roles

```
public JSONObject addMember(Request request, Response
response) {
  var json = new JSONObject(request.body());
  var spaceId = Long.parseLong(request.params(":spaceId"));
  var userToAdd = json.getString("username");
  var role = json.optString("role", "member");    #A

  if (!DEFINED_ROLES.contains(role)) {    #A
    throw new IllegalArgumentException("invalid role");    #A
  }
```